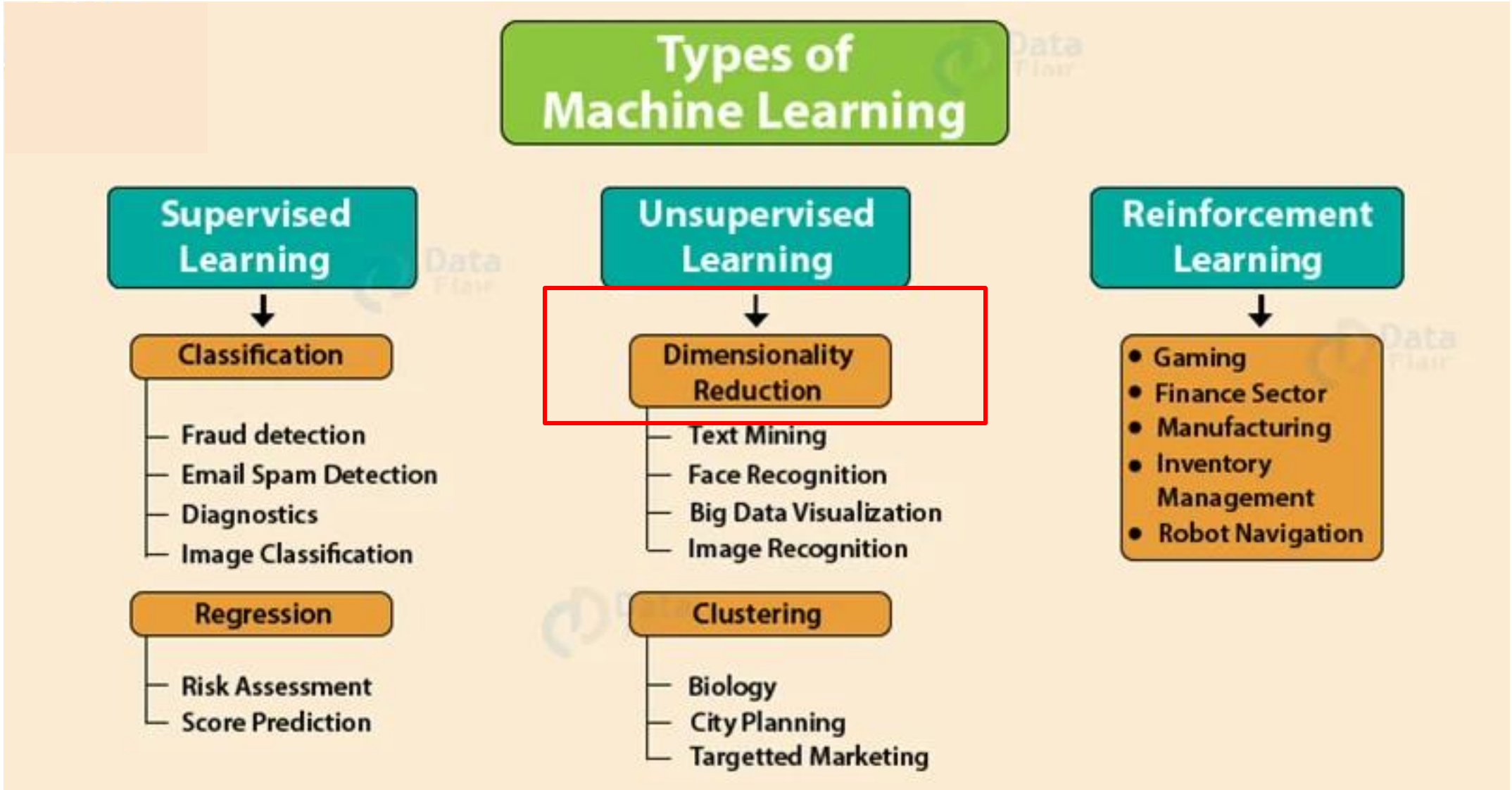
PCA

Principal Component Analysis

A unsupervised learning- dimensionality reduction technique


Types of Machine Learning
Supervised Learning
Classification
Fraud detection
Email Spam Detection
Diagnostics
Image Classification
Regression
Risk Assessment
Score Prediction
Unsupervised Learning
Dimensionality Reduction
Text Mining
Face Recognition
Big Data Visualization
Image Recognition
Clustering
Biology
City Planning
Targetted Marketing
Reinforcement Learning
Gaming
Finance Sector
Manufacturing
Inventory Management
Robot Navigation

Too much of anything is good for nothing!

What happens when the given data set has too many variables?

What if most of the variables are correlated?

What happens to the accuracy of the model if the variables are correlated?

How can we overcome the situation?

Solution: Dimensionality Reduction

The process of **reducing** the number of features (dimensions) in a dataset while **preserving its essential information**.

Basics of Dimensionality Reduction

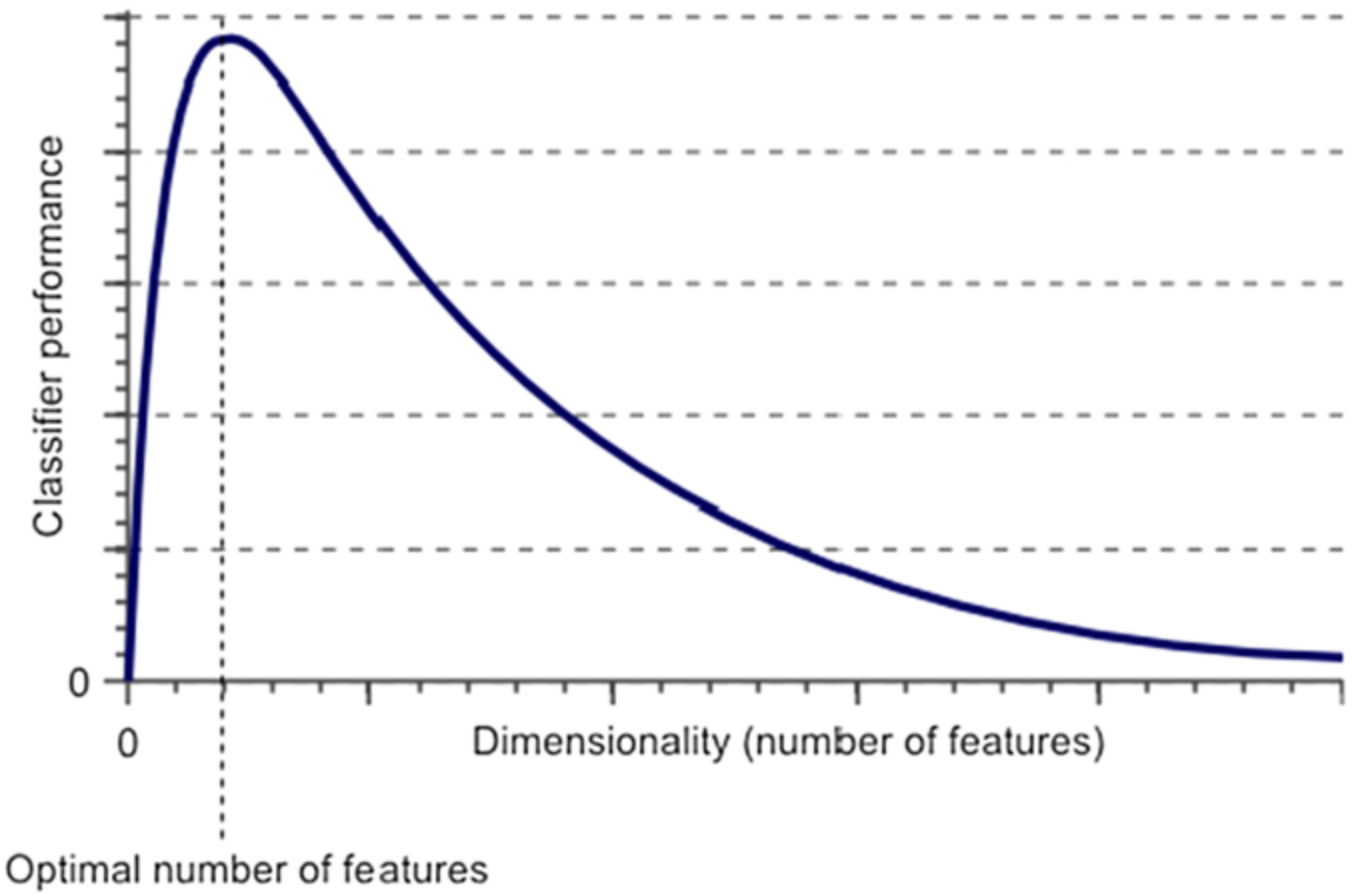
Dimensionality → the number of features/columns/variables in a dataset.The objective is to *minimize this number*.

Why?

- **Redundancy:** Correlated features add noise (e.g., "total purchase amount" vs. "number of items purchased").
- **Efficiency:** Fewer features improve computational efficiency and model performance.
- **Curse of Dimensionality:** Higher dimensions result in sparser data, complicating analysis.

Reducing dimensionality leads to clearer insights and better models. Dimensionality reduction methods are crucial for simplifying models and **preventing overfitting**.

According to Hughes phenomenon, If the number of training samples is fixed and we keep on *increasing the number of dimensions* then the predictive power of our machine learning model first increases, but after a certain point it tends to decrease.

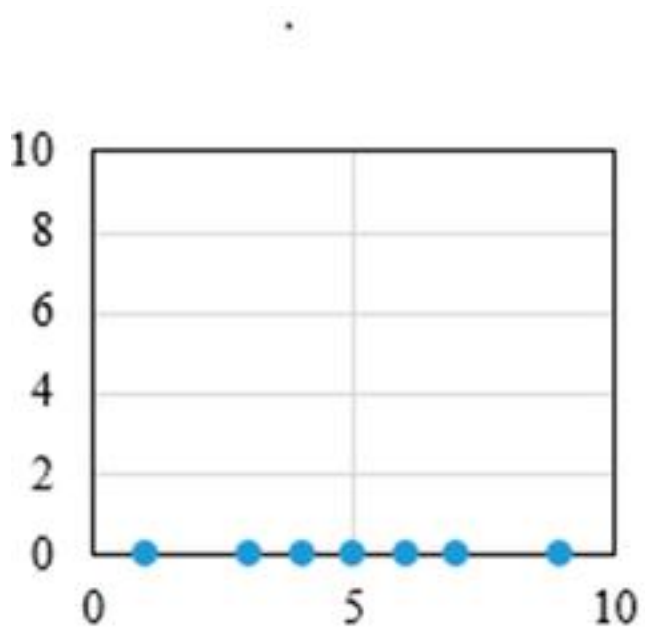


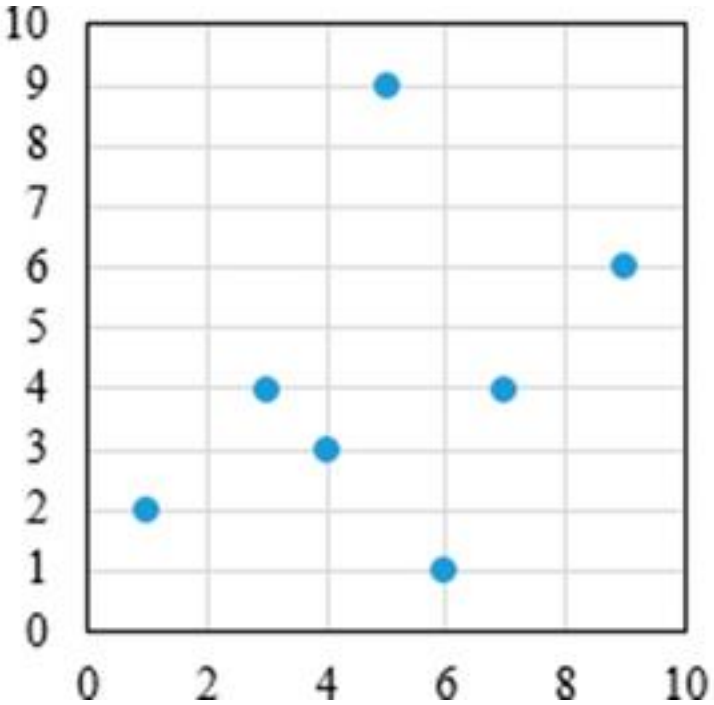
This is known as "The curse of dimensionality".

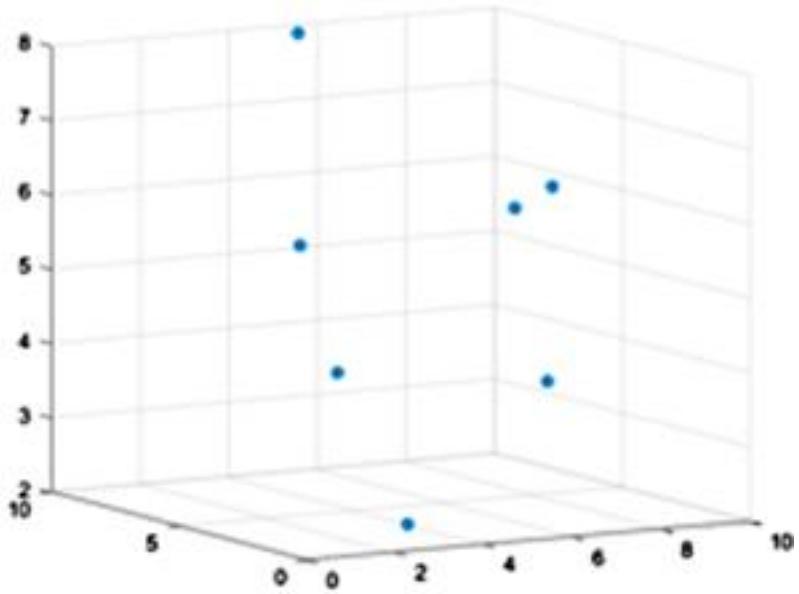
Curse of Dimensionality

Imagine you have a dataset with just a few dimensions, like a list of people's ages and heights. It's easy to visualize and understand this data because it's simple and intuitive.

Now, let's say you **add more dimensions to the dataset, like weights, incomes, and geographical locations.** As the number of dimensions increases, the space in which the data points exist becomes larger and more spread out.



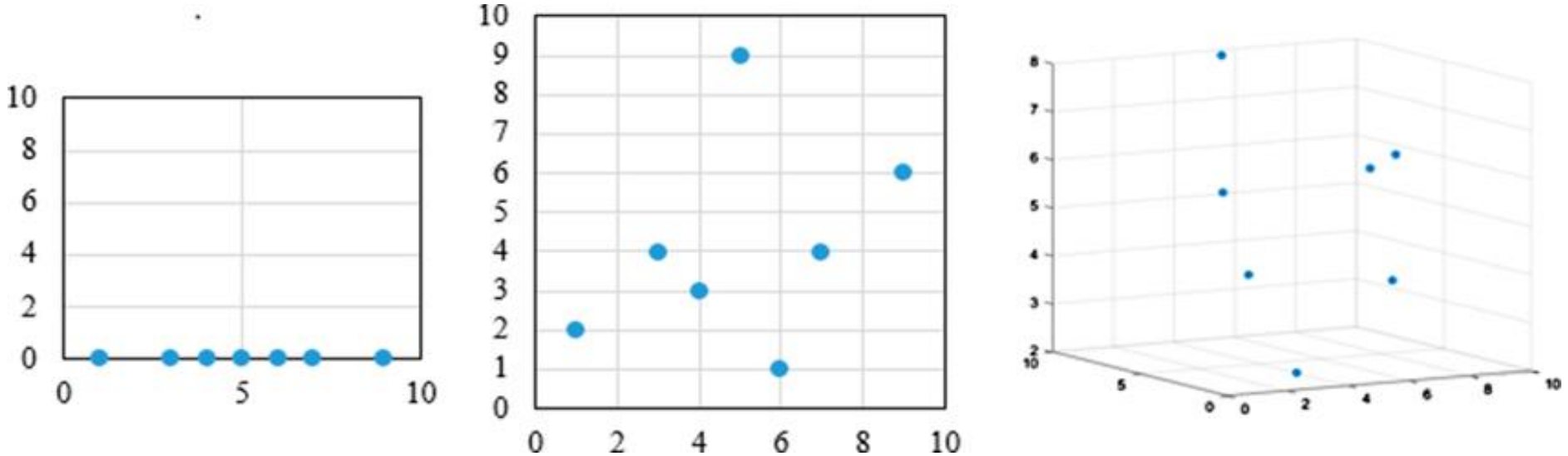




Curse of Dimensionality

Create Data Sparsity: The available data becomes **sparser, meaning there are fewer data points relative to the size of the space.** This makes it harder to find meaningful patterns or relationships in the data.
Also, increased computational complexity, and the risk of overfitting

EXAMPLE:
Imagine distributing **100 data points** in a one-dimensional line (0 to 100).
The points would be relatively close together.
If those same 100 points are placed **in a two-dimensional square**, they become more spread out.
In a **three-dimensional cube**, they are even further apart.
In a **100-dimensional hypercube**, they are extremely isolated from one another.



PRINCIPAL COMPONENT ANALYSIS (PCA)

Widely used method for **unsupervised, dimensionality reduction** machine learning method used to simplify a large data set into a smaller set while still maintaining **significant patterns and trends**.

What is PCA?

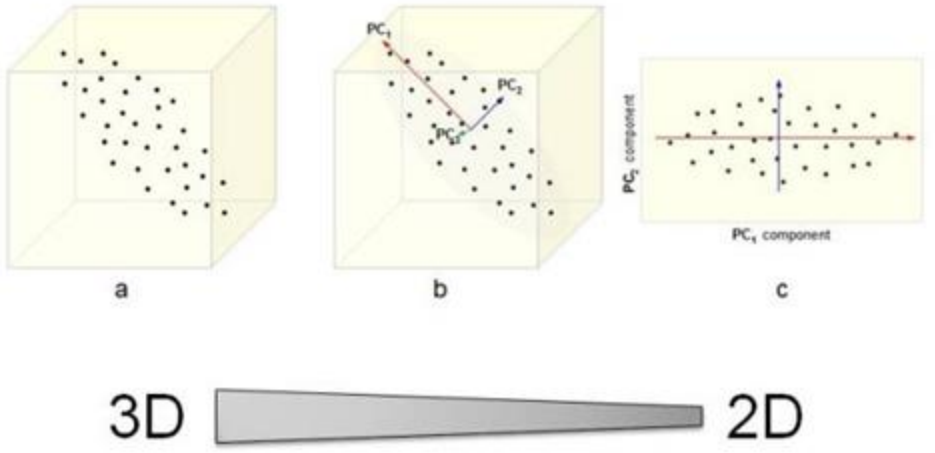
An unsupervised machine learning technique used to reduce the dimensionality of large datasets while preserving **most of the variance**.

Key Idea:

Transforms **correlated variables** into uncorrelated principal components (PCs).

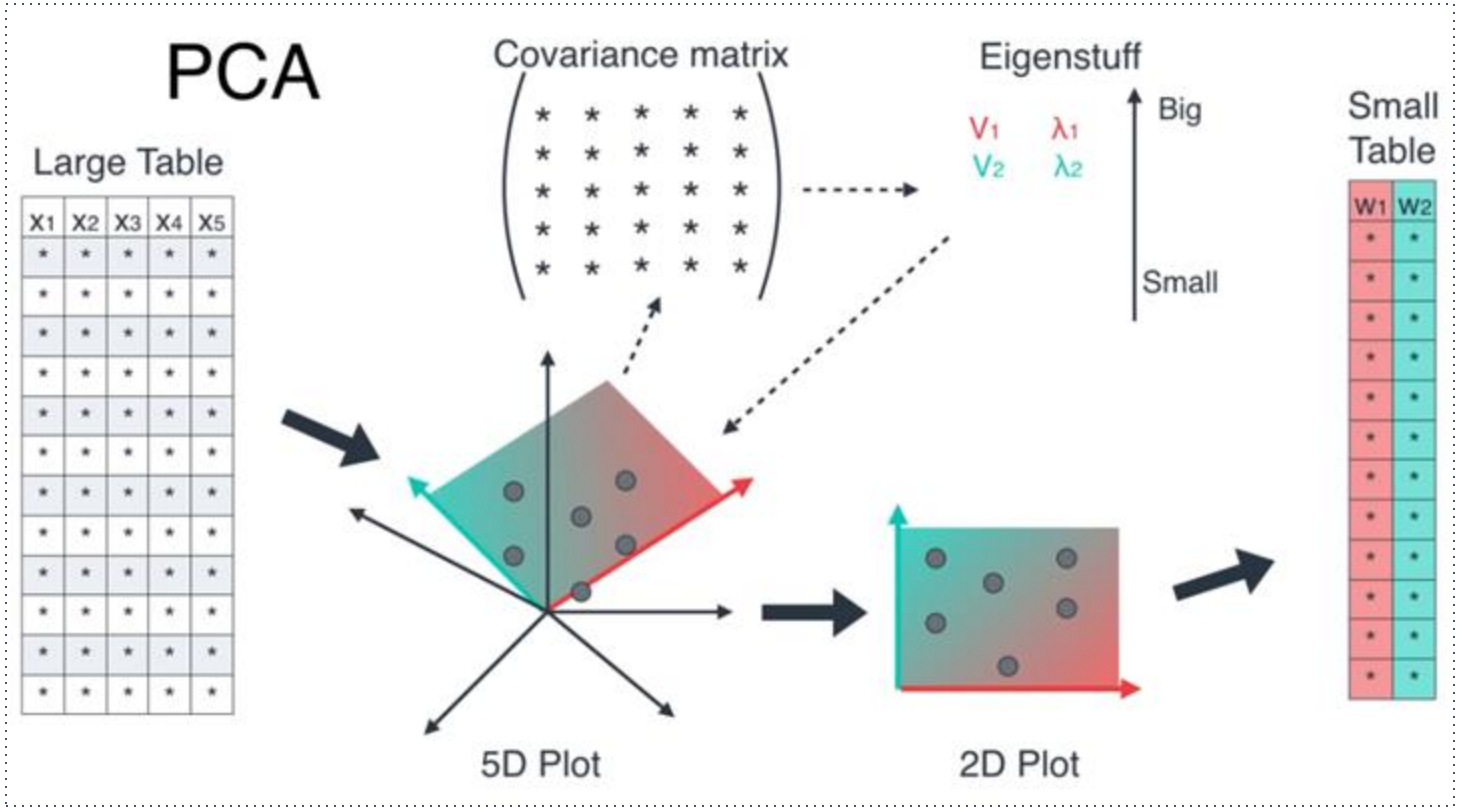
Analogy:

"Viewing data from the most informative angle."



PCA (Principal Component Analysis)

Reduce the number of features (columns or dimensions) in your data **while keeping as much useful information** as possible.



Why Use PCA?

- Helps visualize high-dimensional data (e.g., 100 features → 2 or 3 dimensions).
- Speeds up machine learning models by removing unnecessary features.
- Removes redundancy (correlated features).

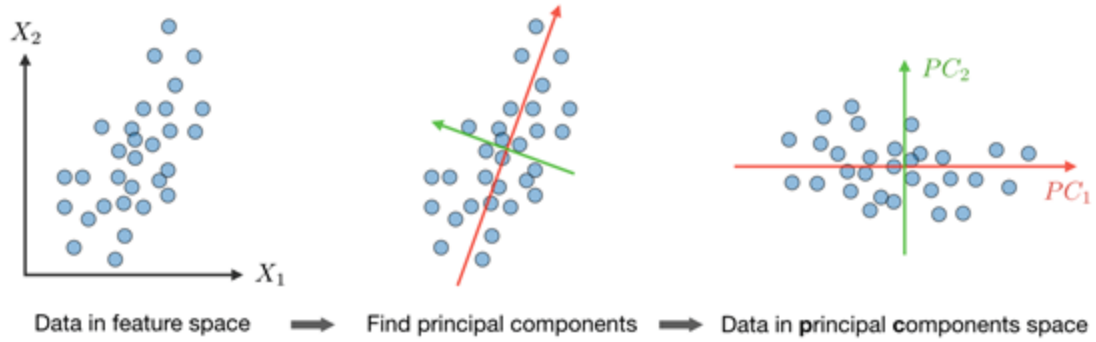
Goal of PCA

(PCA) is known as a **dimension reduction technique** that projects the data on

K dimensions by maximizing the variance of the data.

Goal:

While the data in the **higher-dimensional** space is mapped to data in the **lower-dimensional** space, the **variance of data** in the lower-dimensional space should be maximized.



PCA converts a set of **correlated features** to a set of **uncorrelated features**.

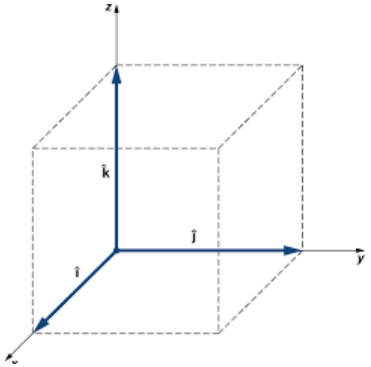
Overview of PCA

A dimensionality reduction technique that:

- Maximizes variance in the projected lower-dimensional space.
- performs **feature extraction** by transforming the original features into new, uncorrelated features (principal components).

Key Idea:

- Projects data onto K orthogonal axes (principal components PC) where variance is maximized.
- PC Preserves the most important structure while reducing noise/redundancy.



Mathematically

- **Input:** Correlated features X.
- **Output:** Uncorrelated components Z=XW (where W = eigenvectors of X^TX (data matrix's sum-of-squares matrix)).

Why It Works:

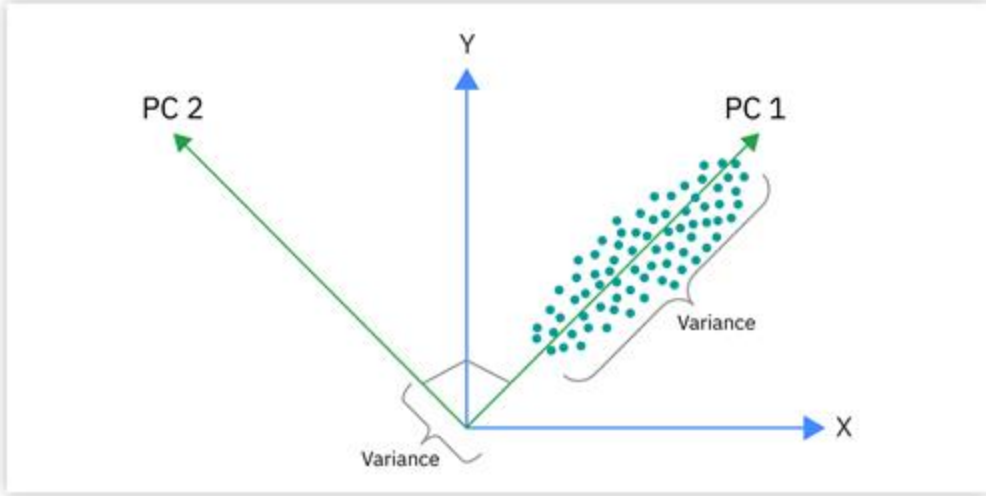
- Eigenvectors = Directions of max variance.
- Eigenvalues = Amount of variance retained.

Key Concepts Behind PCA

Variance Matters: PCs capture directions of maximum variance.

Orthogonality: PCs are perpendicular (uncorrelated).

Eigenvalues: Indicate the importance of each PC.



Feature Extraction using PCA

If we have **N** independent variables. In feature extraction, we create N “new” independent variables, where each “new” independent variable is a combination of each of the N “old” independent variables.

However, we create these new independent variables in a specific way and order these new variables by how well they predict our dependent variable.

Step-by-Step: How PCA Works

1. **Standardize the Data** (Mean = 0, Variance = 1).
2. **Compute Covariance Matrix** (Relationships between features).
3. **Calculate Eigenvectors & Eigenvalues** (Principal Components).
4. **Sort PCs by Eigenvalues** (Highest = Most important).
5. **Project Data onto PCs** (Transform into new space).

Step 1: Standardize the Data

PCA is affected by the scale of features (e.g., height in cm vs. weight in kg).

- Apply "**Standardization**": Convert all features to the same scale (mean = 0, standard deviation = 1).

$$X_{\text{standardized}} = \frac{X - \mu}{\sigma}$$

- μ = mean of the feature
- σ = standard deviation

Why? Ensures all features contribute equally.

Step 2: Compute Covariance Matrix (Σ)

Once the data is standardized, we want to understand **how features vary together**.

Covariance matrix Σ is a table showing how every feature relates to every other feature:

- **Diagonal entries:** Variance of each feature (spread).
- **Off-diagonal entries:** Covariance between feature pairs (relationships).
  - Positive → Features increase together
  - Negative → One increases while the other decreases

For standardized data (mean=0, std=1):

$$\Sigma = \frac{1}{n} X^T X$$

where:

- X = Standardized data matrix (rows: samples, columns: features)
- n = Number of samples

$$X^T X = \begin{bmatrix} \text{Feature 1} \cdot \text{Feature 1} & \text{Feature 1} \cdot \text{Feature 2} & \cdots \\ \text{Feature 2} \cdot \text{Feature 1} & \text{Feature 2} \cdot \text{Feature 2} & \cdots \\ \vdots & \vdots & \ddots \end{bmatrix}$$

Each entry cov(xi,xj) represents how features i and j vary together.

Structure (for 3 features):

$$\Sigma = \begin{bmatrix} \text{Var}(X_1) & \text{Cov}(X_1, X_2) & \text{Cov}(X_1, X_3) \\ \text{Cov}(X_2, X_1) & \text{Var}(X_2) & \text{Cov}(X_2, X_3) \\ \text{Cov}(X_3, X_1) & \text{Cov}(X_3, X_2) & \text{Var}(X_3) \end{bmatrix}$$

Why? We want to find directions in feature space where the data varies the most — and the covariance matrix tells us that.

Step 03: Calculate the Eigenvectors and Eigenvalues of the Covariance Matrix

Next we decompose the covariance matrix into eigenvalues and eigenvectors.

- Eigenvectors and eigenvalues are the linear algebra concepts that we need to compute from the covariance matrix to determine the principal components of the data.

Principal Components: By finding the eigenvector with the largest eigenvalue, we identify the direction of the largest spread in the data.

Eigenvectors and Eigenvalues

Covariance Matrix defines both the spread (variance) and the orientation (covariance) of our data.

- Eigenvectors: Indicate the directions of maximum variance (most information) in the data (Principal Components).
  - Why? the eigenvectors of the covariance matrix are the directions where variance is maximized.
- Eigenvalues: are simply the coefficients attached to eigenvectors, which give the amount of variance carried in each Principal Component.
  - Represent the magnitude of variance in each Principal Component.

Step 3: Find Eigenvectors & Eigenvalues

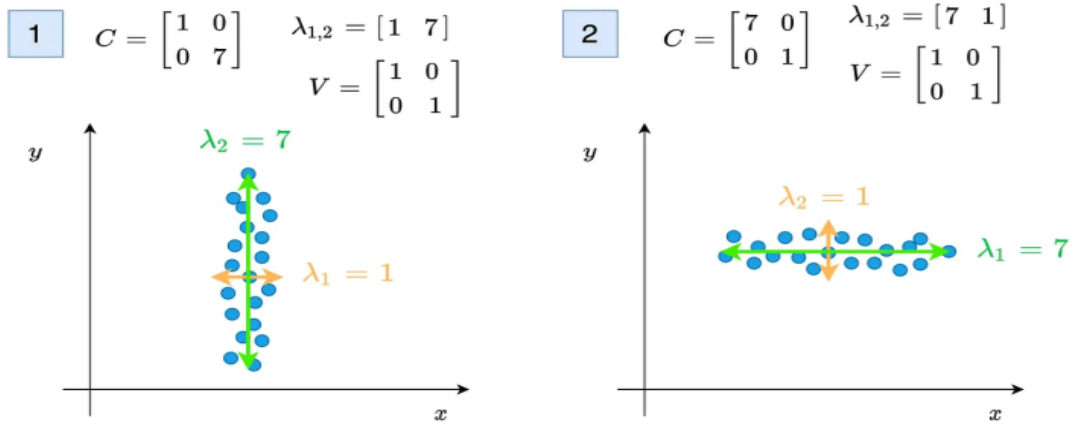
PCA uses Σ to: **Find** directions (eigenvectors) of max variance and **Determine** importance (eigenvalues) of each direction.

We solve: $\Sigma v = \lambda v$

- Σ (Sigma) = Covariance matrix (shows how features relate to each other)
- v = **Eigenvector** (direction (Principal Component))
- λ (Lambda) = An **eigenvalue** (a number/scalar telling how important (variance) that direction is).
  - **PCs are sorted by λ: biggest λ = most important**

What we get?

- ❑ All the eigenvectors together form new axes/directions in feature space — these are the **principal components**.
  - ❑ 1st eigenvector = direction of max variance
  - ❑ 2nd eigenvector = next best **orthogonal** direction
- ❑ We sort them by eigenvalues (largest first).



Step 4 (Part A): Sort/Rank all Principal Components

Sort eigenvalues in descending order (highest to lowest).

Step 1: Sort Eigenvalues:

- Arrange eigenvalues (λ) in descending order: λ1> λ2> …>λn (Largest λ = direction of max variance)

Step 2: Choose Top *k* Eigenvectors

- Select the first *k* eigenvectors (PCs) corresponding to the largest eigenvalues. These define the most important directions in the data.

NEXT: Step 3: Determine *k* (**How Many PCs to Keep?**)

Eigenvectors (v) = Best directions to project data.

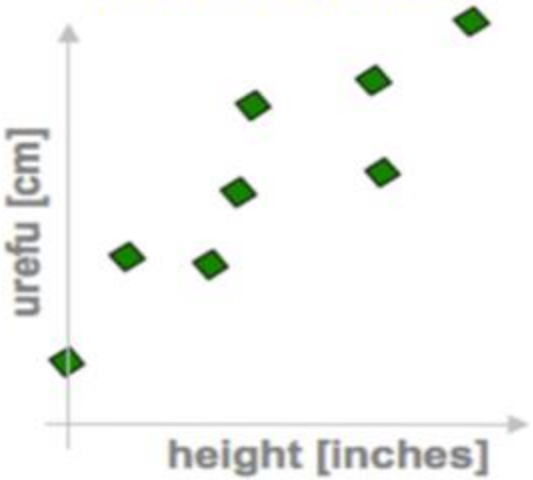
Eigenvalues (λ) = How important each direction is.

PCA uses this to find the "most important" directions (principal components) and throw away low-variance directions to reduce dimensions.

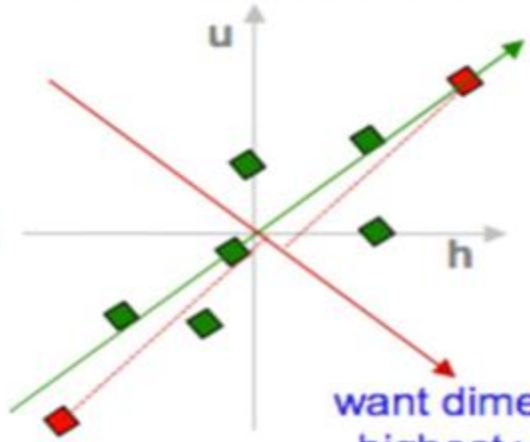
PCA in a nutshell

1. correlated hi-d data

("urefu" means "height" in Swahili)



2. center the points



want dimension of highest variance

3. compute covariance matrix

	h	u
h	2.0	0.8
u	0.8	0.6

$$\text{cov}(h,u) = \frac{1}{n}\sum_{i=1}^{n} h_i u_i$$

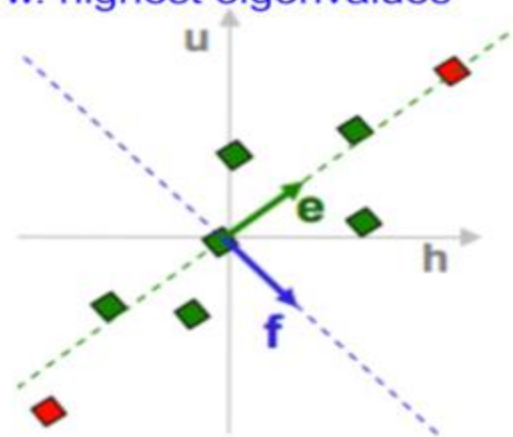
4. eigenvectors + eigenvalues

$$\begin{pmatrix} 2.0 & 0.8 \\ 0.8 & 0.6 \end{pmatrix} \begin{pmatrix} e_h \\ e_u \end{pmatrix} = \lambda_e \begin{pmatrix} e_h \\ e_u \end{pmatrix}$$

$$\begin{pmatrix} 2.0 & 0.8 \\ 0.8 & 0.6 \end{pmatrix} \begin{pmatrix} f_h \\ f_u \end{pmatrix} = \lambda_f \begin{pmatrix} f_h \\ f_u \end{pmatrix}$$

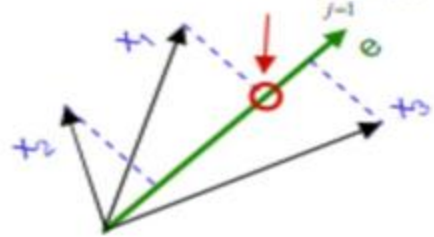
```
eig(cov(data))
```

5. pick m<d eigenvectors w. highest eigenvalues

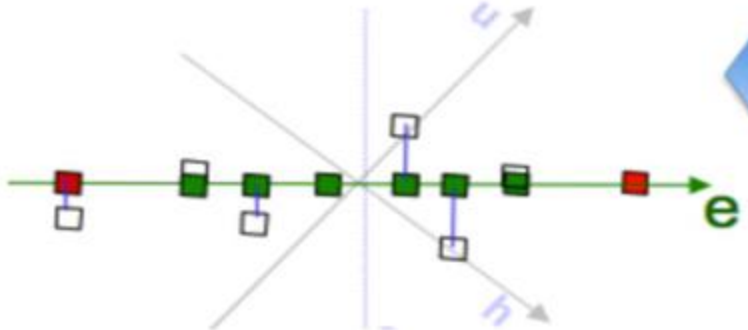


6. project data points to those eigenvectors

$$x_e' = x^T e = \sum_{j=1}^{d} x_j e_j$$



7. uncorrelated low-d data



Copyright © 2011 Victor Lavrenko

Step 4 (Part B): Select Top Principal Components

How to decide how many PCs to keep, look at each **eigenvalue's contribution** to total variance.

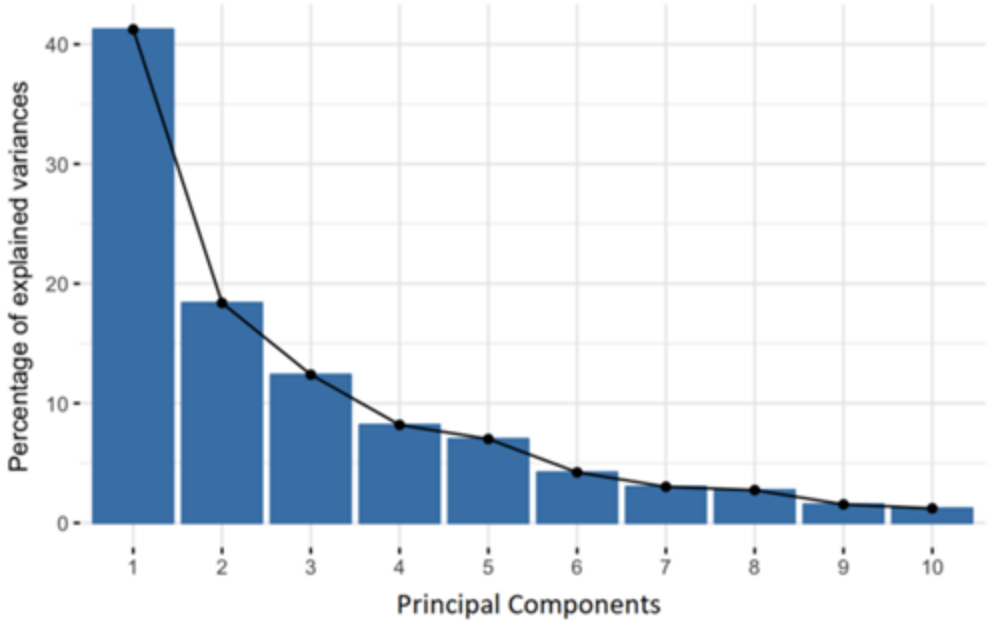
Choose the **top k** eigenvectors (principal components) that capture most of the variance.

How many components (k) to keep?

- We keep the PCs that collectively capture most of the variance and discard those with minimal contribution.
- Use the explained variance ratio:

$$\text{Explained Variance} = \frac{\lambda_i}{\sum \lambda}$$

Keep enough components to retain, say, **95%** of total variance.

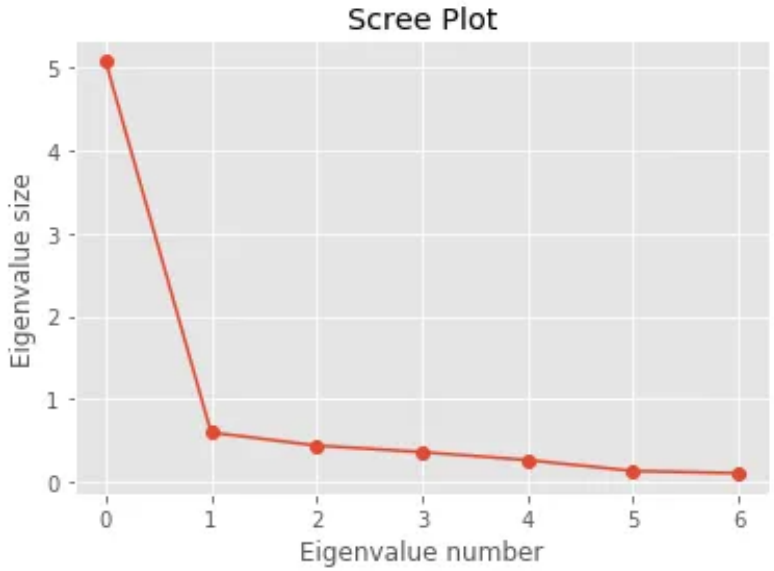


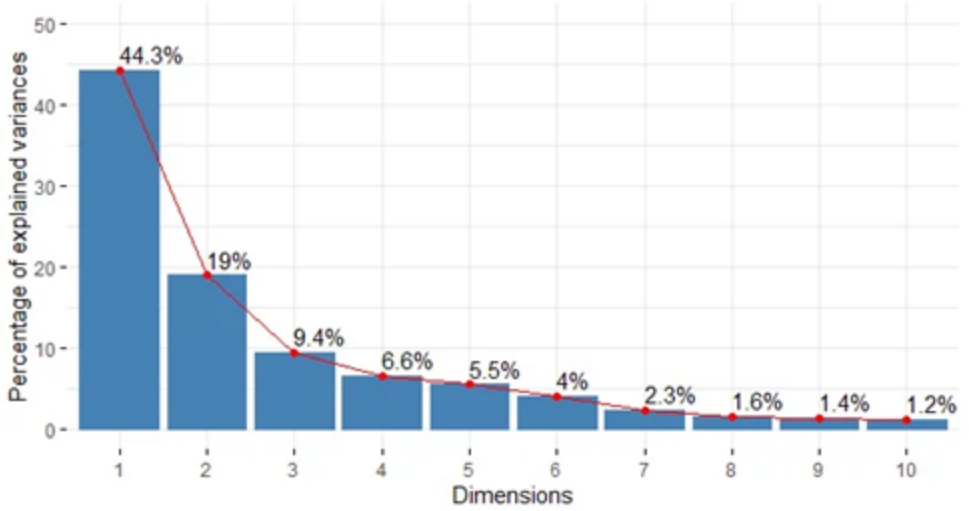
Common strategies:

- Retain PCs covering >95% cumulative variance (for accuracy).
- Keep PCs with $\lambda \geq 1$ (Kaiser's rule).
- Use the elbow method in a scree plot.

How to decide top **k** PCs

1. Elbow Method: Create a Scree plot
2. For data visualization: select 2 or 3 PCs.
3. Calculate the percentage of explained variance and analyze the contribution percentages:
    - **Common approach:** keep PCs that together explain a specific percentage of the total variance (e.g., 80% or 90%).





Each bar shows the explained variance percentage of individual components. The red line is a type of scree plot. By analyzing this plot, we decide to keep only the first two or three components because they capture almost all the variance in the dataset.

Step 5: Transform the Data: Project Onto New Axes

Finally, we project our original standardized data $X_{standardized}$ onto the selected eigenvectors W_k:

$$X_{reduced} = X_{standardized} \cdot W_k$$

matrix of top k eigenvectors (each column is an eigenvector).

This gives us a new dataset with fewer dimensions (say, 2 instead of 5), where:

- Each column is a principal component,
- Most variance is retained.
- **Result:** Original data (e.g., 10 features) → Reduced data (e.g., 2 features).

PCA in Python (sklearn)

```python
import numpy as np
from sklearn.decomposition import PCA
from sklearn.preprocessing import StandardScaler
import matplotlib.pyplot as plt

# Example Data
np.random.seed(42)
X = np.random.rand(100, 3)  # 100 samples with 3 features

# Step 1: Standardize the Data
scaler = StandardScaler()
X_std = scaler.fit_transform(X)

# Step 2-5: PCA
pca = PCA()
X_pca = pca.fit_transform(X_std)

# Plot Explained Variance Ratio
explained_var_ratio = pca.explained_variance_ratio_
cumulative_var_ratio = np.cumsum(explained_var_ratio)

plt.plot(range(1, len(cumulative_var_ratio) + 1), cumulative_var_ratio, marker='o')
plt.xlabel('Number of Principal Components')
plt.ylabel('Cumulative Explained Variance Ratio')
plt.title('Explained Variance Ratio vs. Number of Principal Components')
plt.show()
```

Key Takeaways

- PCA = Find new axes (PCs) where data varies most.
- Eigenvectors = directions, Eigenvalues = importance (how much variance is in that direction).
- PCA chooses the directions that best explain the data.
- Project data onto top PCs to reduce dimensions.
- PCA is especially useful for preprocessing, noise reduction, and exploration.

Benefits	Downsides to PCA
Dimension Reduction: PCA reduces the number of features while retaining essential information, making datasets more manageable. **Noise Reduction:** eliminate noise in data, improving feature quality and model performance. **Feature Transformation:** transforms original features into orthogonal components, potentially revealing hidden patterns. **Visualization:** PCA simplifies data into 2 or 3D for easier visual understanding. **Feature Selection:** PCA ranks principal components, helping indirectly in feature selection.	**Linearity:** PCA assumes straight-line relationships in data, which isn't always the case. **Interpretability:** Principal components can be hard to understand in terms of original features. **Loss of Information:** PCA prioritizes variance and might lose important data details. **No Feature Learning:** PCA doesn't create new features; it works with what's already there.